

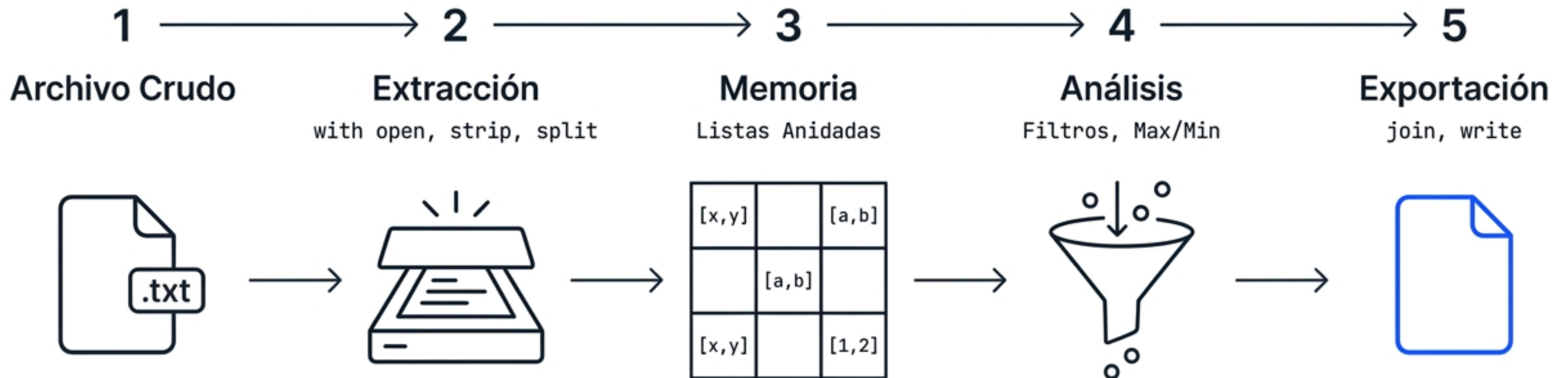


De Texto a Insights

Dominando Archivos, Cadenas y Listas en Python

El Pipeline de Procesamiento de Datos

Procesar información plana requiere un flujo metódico. Transformamos secuencias de caracteres estáticas en estructuras dinámicas para aislar información crítica y generar nuevos reportes.



La Puerta Automática: with open

El Context Manager (**with open**) es el estándar de Clean Code para manejar archivos. Garantiza la liberación de recursos en memoria automáticamente, incluso si ocurre un error durante la lectura.

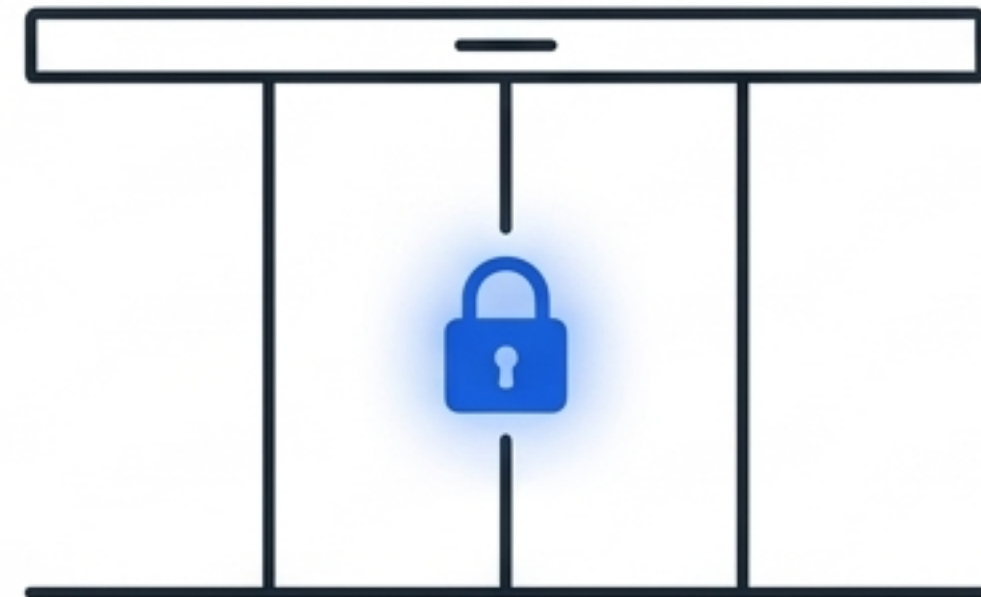
The Legacy Method

```
f = open('datos.txt')  
...  
f.close()
```



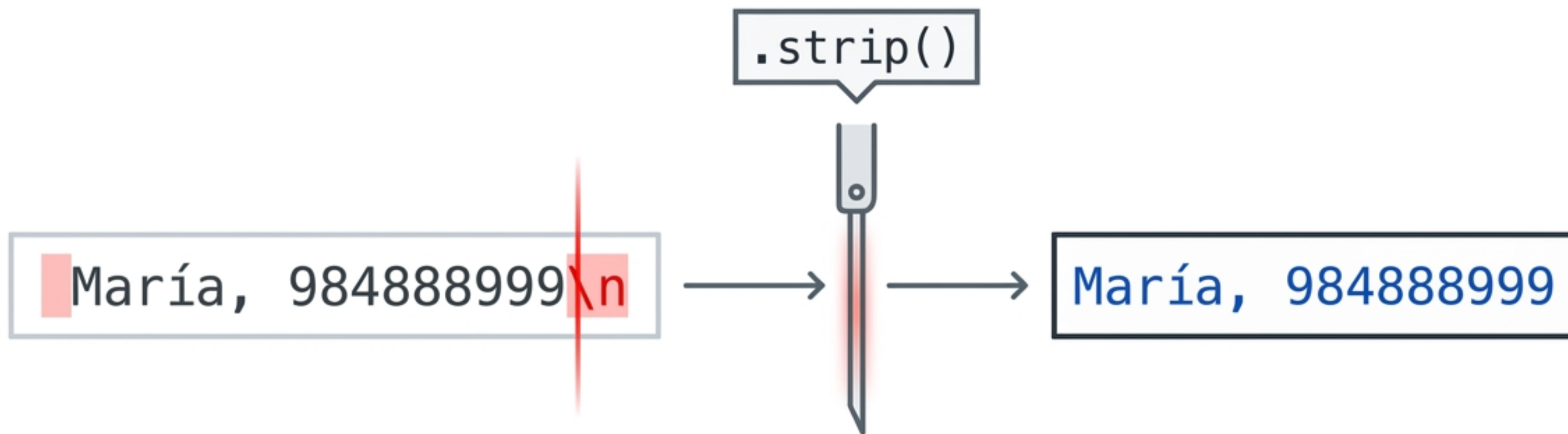
The Clean Code Method

```
with open('datos.txt', 'r') as f:  
    for linea in f:  
        procesar(linea)
```



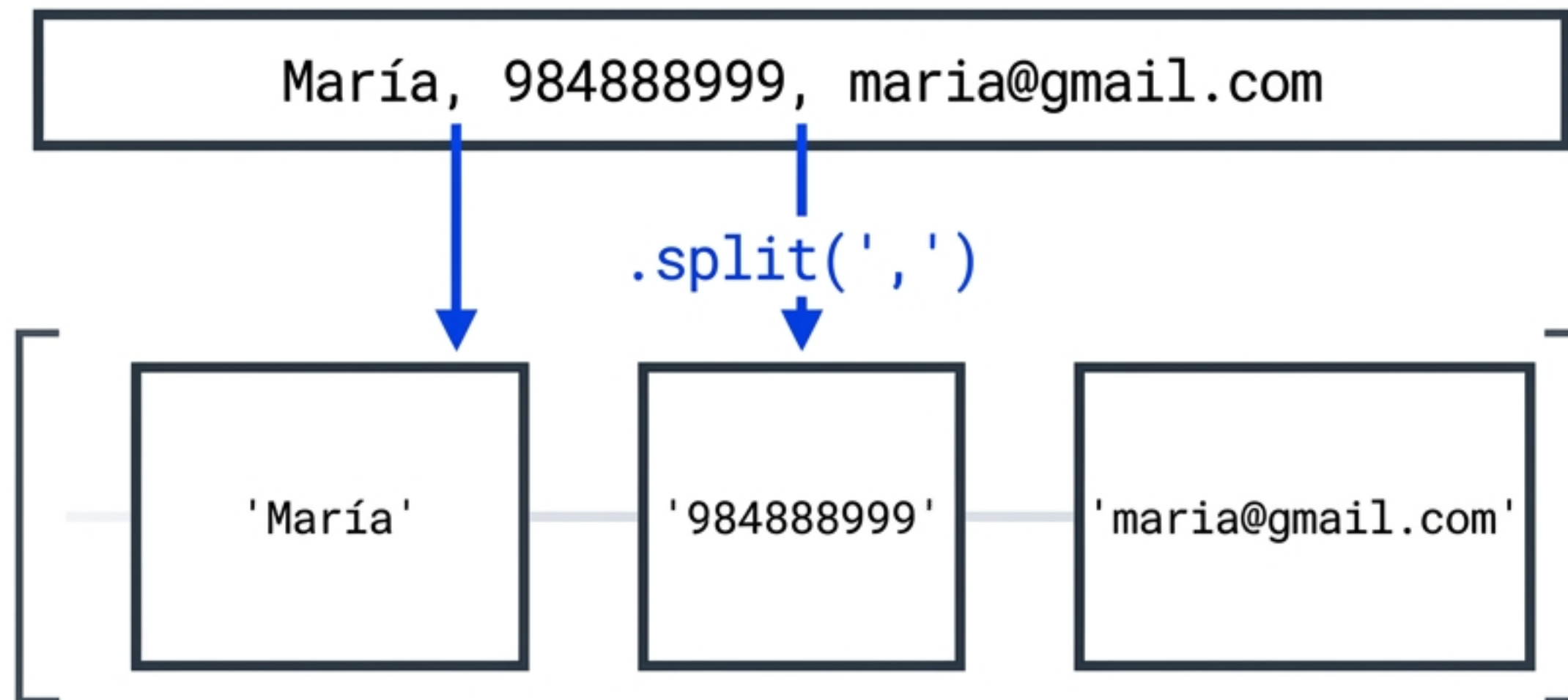
Purificando la Entrada: `.strip()`

Los archivos de texto contienen metadatos invisibles como saltos de línea (`\n`) o espacios residuales. **`.strip()`** actúa como un bisturí perimetral, eliminando ruido antes del análisis.



El Bisturí de Cadenas: `.split()`

El texto plano no es analizable. `.split()` segmenta una cadena de texto utilizando un delimitador específico (comas, punto y coma, espacios) y la convierte en una Lista indexable.



Construyendo la Base de Datos Interna

Al iterar sobre el archivo, aplicamos limpieza y corte línea por línea. El resultado se inyecta (.append) en una lista principal, generando una matriz bidimensional (Lista de Listas) lista para el análisis.

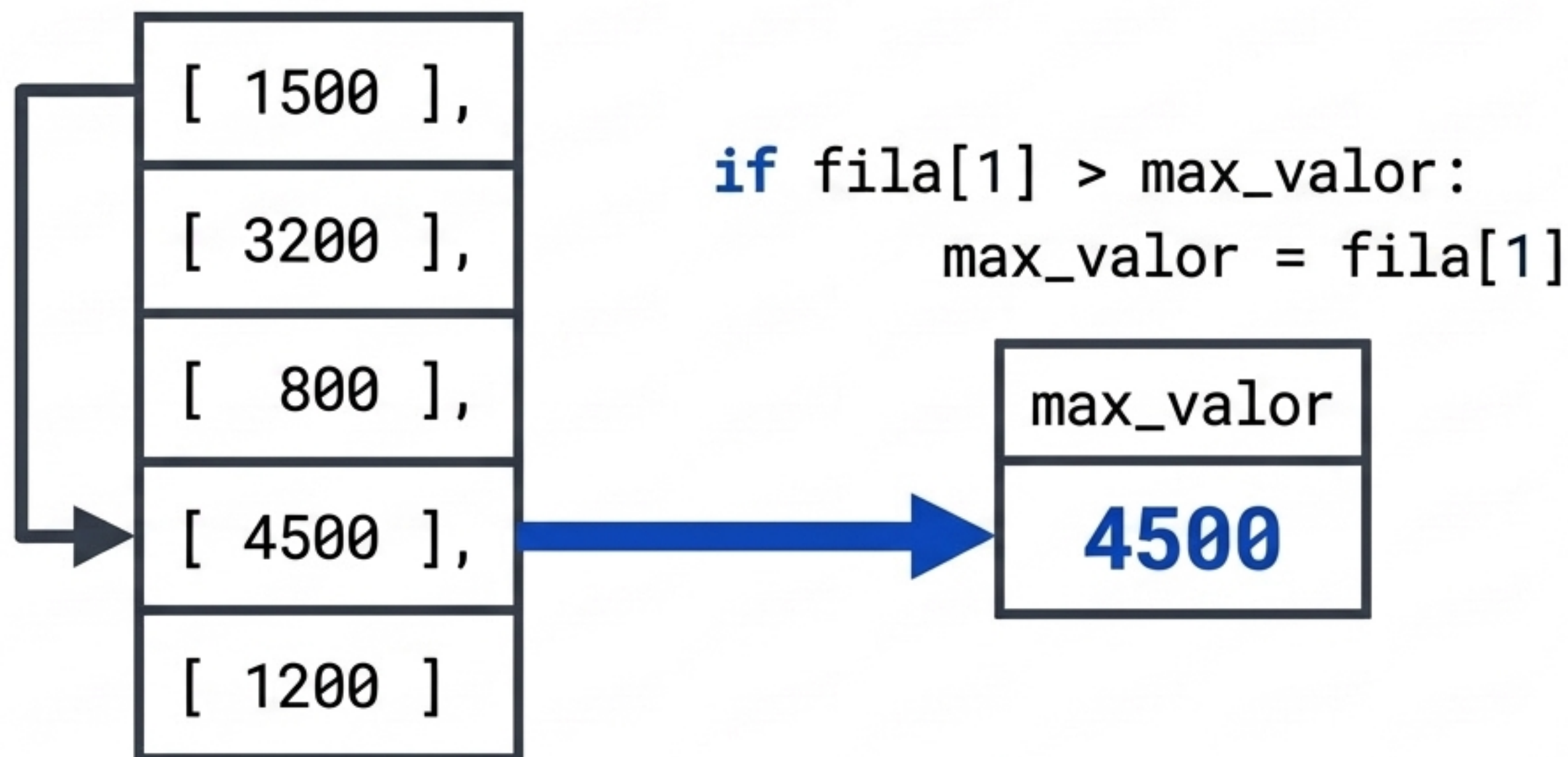
```
contactos = []  
for linea in f:  
    datos = linea.strip().split(',')  
    contactos.append(datos)
```



	[0]	[1]	[2]
[0]	['María']	'984888999'	'maria@gmail.com']
[1]	['Juan']	'984111222'	'juan@yahoo.com']
[2]	['Ana']	'984333444'	'ana@outlook.com']

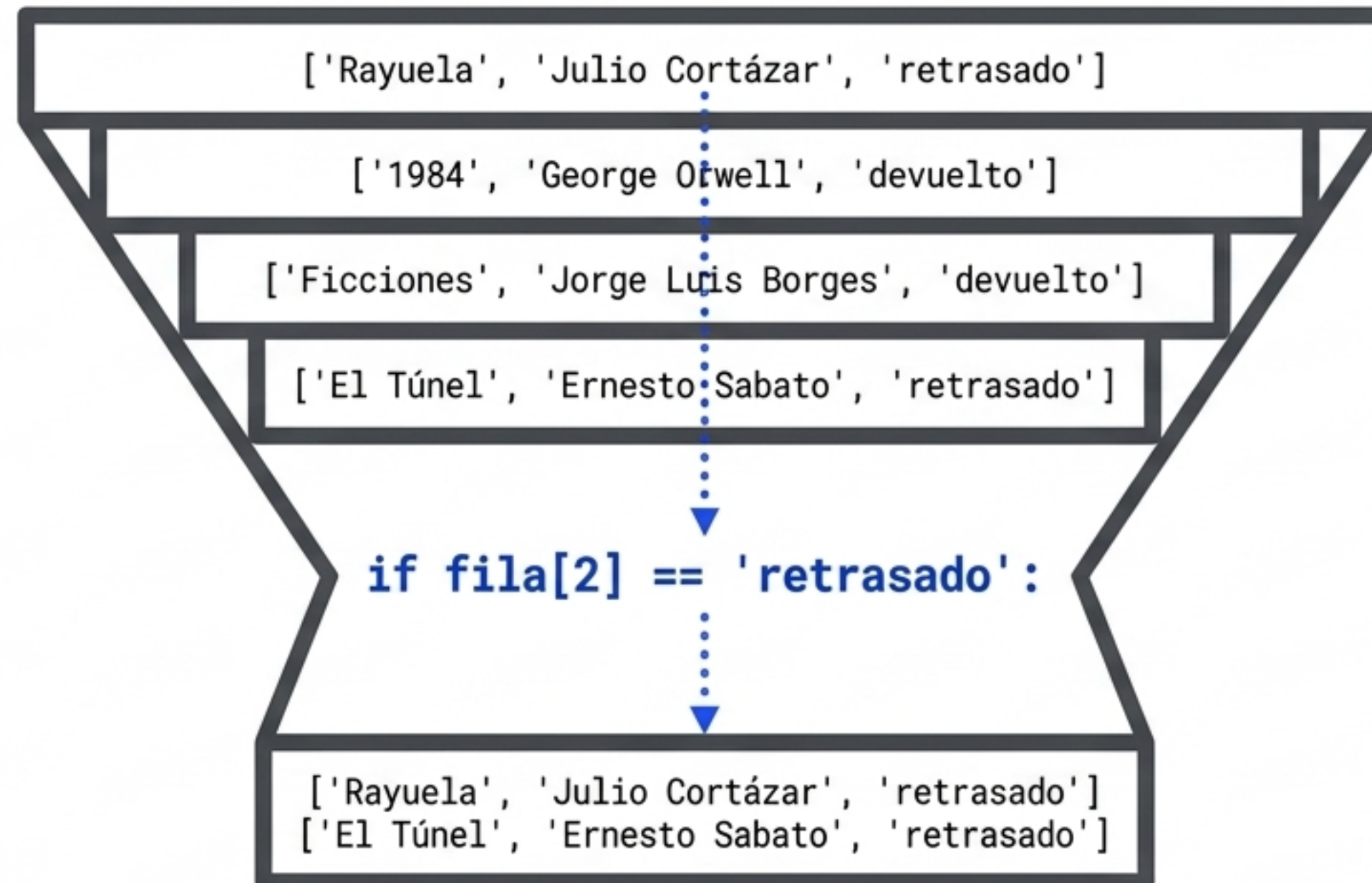
Buscando los Extremos: Máximos y Mínimos

Para encontrar el 'mayor' o 'menor' (ej. usuario con más préstamos o producto más caro), iteramos nuestra matriz comparando una columna específica contra una variable de control inicializada estratégicamente.



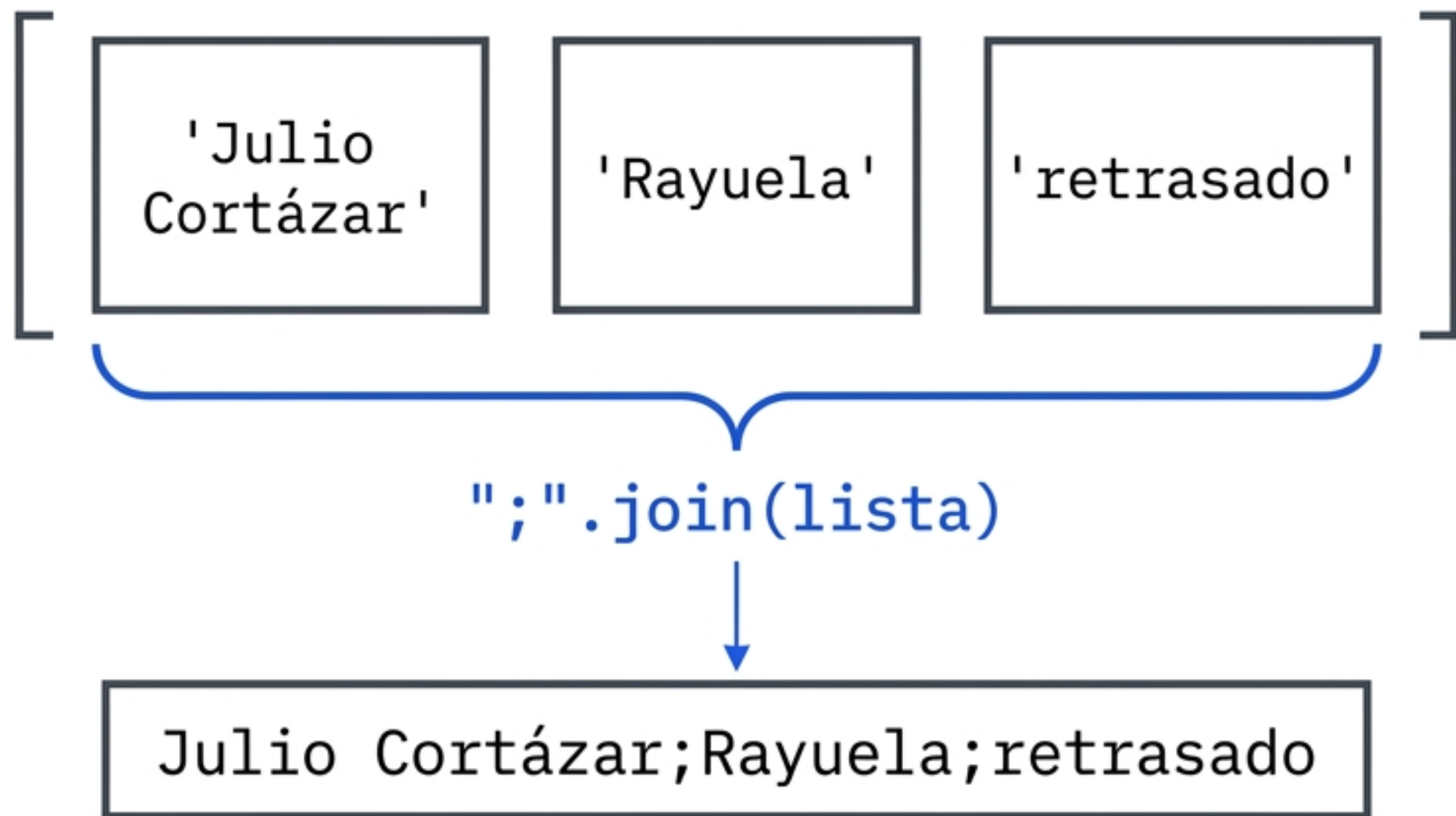
Filtrado Inteligente de Datos

El análisis frecuentemente requiere aislar subconjuntos de información. Accediendo a los índices específicos de cada sublista (fila[i]), evaluamos condiciones lógicas para extraer únicamente los registros relevantes.



El Ensamblador: .join()

Antes de exportar, los datos estructurados en listas deben aplanarse. `.join()` actúa como pegamento, uniendo los elementos de una lista mediante un delimitador para reconstruir la cadena de texto.



Generando el Reporte de Salida

Utilizando el modo de escritura ('w'), abrimos un nuevo lienzo. Iteramos sobre nuestra lista analizada, ensamblamos la cadena con `.join()`, añadimos el salto de línea (`\n`), y grabamos los insights.

```
with open('informe.txt', 'w') as f_out:  
    for registro in filtrados:  
        linea = ';'.join(registro) + '\n'  
        f_out.write(linea)
```



informe.txt



Reto de Negocio: Gestión de Préstamos

Examen Práctico: Se requiere leer el archivo prestamos.txt, excluir al usuario administrador, listar los libros no devueltos, y generar un informe automatizado por autor (informe_[autor].txt).

Estructura del Archivo:

fecha ; libro ; autor ; usuario ; estado

- Extraer datos a Memoria (Lista de Listas)
- Excluir 'administrador'
- Encontrar libros NO devueltos
- Buscar autores principales

```
2025-09-01; Rayuela; Julio Cortázar; Ana Gutierrez; devuelto
2025-09-02; Cien años de soledad; Gabriel García Marquez; administrador; pendiente
2025-09-03; El Extranjero; Albert Camus; Carlos Gómez; retrasado
```

Fase 1: Extracción y Filtrado Base

Aplicamos with open y .split(';'). Durante la ingesta, incluimos una validación lógica simple (if fila[3] != 'administrador':) para limpiar la base de datos en memoria desde el primer paso.

```
for linea in f:  
    fila = linea.strip().split(';')  
    if fila[3] != 'administrador' and  
    if fila[3] != 'administrador':  
        prestamos_lista.append(fila)
```



~~Gabriel García Marquez | administrador~~

Ana Gutierrez	devuelto
Carlos Gómez	retrasado

Fase 2: Análisis y Cruce de Datos

Con la matriz en memoria, iteramos para extraer vectores específicos. Filtramos por el índice del estado (índice 4) y agregamos conteos por el índice de usuario (índice 3) para encontrar los valores máximos.

No Devueltos

```
if estado != 'devuelto':
```

- Carlos Gómez [**retrasado**]
- Maria Paz [**pendiente**]

Top Usuarios

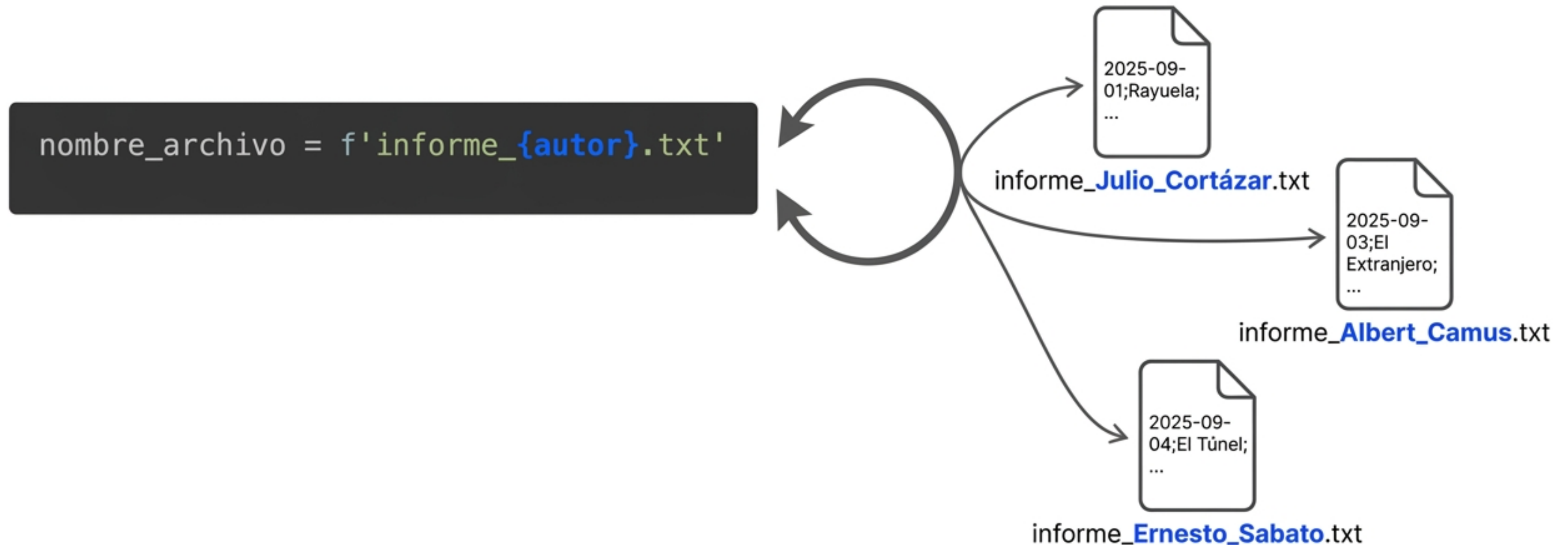
User	Count
Carlos Gómez	5
Ana Gutierrez	2
Luis Silva	8

max_valor

Usuario TOP:
Luis Silva (8)

Fase 3: Generación Dinámica de Informes

El punto obligatorio exige informes personalizados. Usamos f-strings para nombrar los archivos de salida dinámicamente según el autor, uniendo (join) los registros correspondientes y escribiéndolos en disco.



El Arte del Procesamiento Plano

Dominar el manejo de archivos de texto en Python es comprender este flujo. Con solo un puñado de métodos (`open`, `strip`, `split`, `join`) y estructuras de listas, cualquier formato plano puede convertirse en inteligencia de negocio.

EXTRACT

```
prestamos = []  
with open('prestamos.txt', 'r') as f:  
    for linea in f:  
        fila = linea.strip().split(';')  
        if fila[3] != 'administrador':  
            prestamos.append(fila)
```

TRANSFORM

```
no_devueltos = []  
for p in prestamos:  
    if p[4] != 'devuelto':  
        no_devueltos.append(p)
```

LOAD

```
with open('informe_final.txt', 'w') as f_out:  
    for registro in no_devueltos:  
        f_out.write(';'.join(registro) + '\n')
```